Phishing Website Detection using Machine Learning

CPE4221305 — Introduction to Artificial Intelligence, Spring 2026

Ankara Medipol University

Halil Taha Demir
School of Engineering and Natural Sciences
Ankara Medipol University
Ankara, Turkey
Student ID: 64210012
halil.demir@std.ankaramedipol.edu.tr

***Abstract*—In recent years with the increasing technological developments today user information has become much more important. Phishing is one of the primary methods used to obtain this user information through websites. In this project machine learning techniques are employed to detect phishing websites. Decision Tree, Naive Bayes, and k-NN (k-Nearest Neighbors) algorithms were used and compared. There are about 11k samples and 30 different features in the dataset used. The models were compared using accuracy, precision, recall and F1-score metrics. As a result of the evaluation the decision tree algorithm has the best result with an accuracy rate of about 95%. The k-NN algorithm also achieved a high accuracy rate (93%) and was the second best algorithm. The Naive Bayes algorithm on the other hand performed much worse due to the dependencies between features in the dataset and was the worst algorithm among these three (58% accuracy). As a result it has become evident how crucial algorithm selection is in detecting phishing websites and the significant impact this choice has on performance. This study has thus revealed the critical role played by proper model selection in cyber security applications in terms of affecting performance, outcomes and consequently user security.**

***Index Terms*—Artificial Intelligence, Machine Learning, Phishing Detection, Decision Tree, Naive Bayes, k-NN**

I. Introduction

In the last decade, with the widespread use of the internet cyber attacks and cyber threats have increased significantly. Among these threats phishing attacks are one of the most important. The primary purpose of phishing attacks is to obtain users personal and financial information. The most commonly used method for this is through fake websites. Therefore it is necessary to detect phishing websites quickly and accurately and take precautions. Phishing detection is a challenging problem due to attackers constantly developing new methods. Traditional security methods cannot always successfully detect such attacks. Therefore, more flexible and learnable systems are needed. As a result of the development of machine learning, different machine learning algorithms have become methods that offer effective solutions for phishing detection. By using various algorithms, analyses derived from different features of websites determine whether the site is phishing or not. However, the performance of the algorithm used may vary depending on the data set and features. The main objective of this report is to compare the performances of different machine learning algorithms used in the detection of phishing websites. Decision Tree, Naive Bayes, and k-NN algorithms were applied, and the obtained results were analyzed and compared. This report is structured as follows: First the relevant studies are reviewed then the problem definition and methods used are explained. Next the obtained results are presented and discussed; finally, the study's conclusions and potential future improvements are examined.

II. Related Work

The detection of phishing websites has become an important research topic in recent years due to the increasing cyber attacks and cyber threats. There are numerous studies in the literature that use various machine learning techniques to solve this problem.

In the study conducted by Salahdine and his colleagues machine learning-based approaches for detecting phishing attacks were discussed. In this study multiple diverse data sets were used, and meaningful features were extracted from the data. Different machine learning models were trained using these features. In the final case study, the performance of these models was analyzed using metrics such as accuracy, precision, and recall. The results of this study indicate that machine learning methods yield effective results in phishing detection. Additionally it was emphasized that the performance of different algorithms may vary depending on the data set and the features used. [3]

Similarly, Sahoo and his colleagues examined machine learning techniques used for malicious URL detection in detail. As a result of this study, it was concluded that traditional methods are inadequate against current and constantly evolving attack methods. However, it has been observed that machine learning techniques yield more effective results in detecting new and unknown threats. Additionally, it has been observed that different algorithms vary depending on the structure of the data set and the features used. [2]

In this report the aim was to apply different machine learning algorithms on the same data set and compare their performances. For this comparison, three algorithms were selected (Decision Tree, Naïve Bayes, k-Nearest Neighbors) and tested on the same data set. Thanks to this approach,

the performance of algorithms under the same conditions was analyzed in a more objective manner. Additionally as an additional component of this analysis, the advantages and disadvantages of different algorithms when analyzed on the same problem were examined and the differences between them were clearly demonstrated. Thus it was attempted to determine for what types of data sets these algorithms are more suitable to be used. In this respect the study aims to conduct a comparative analysis of different machine learning algorithms.

References : [2], [3]

III. Problem Formulation

A. Formal Problem Definition

In this study, the problem is a machine learning-based classification problem for detecting phishing websites. Phishing attacks are one of the significant cyber threats targeting users financial data and personal information through fake websites. Therefore accurately classifying websites as phishing or legitimate is of great importance for user security.

The problem is a binary classification problem addressed within supervised learning. In this type of learning, the model is trained on previously labeled data and then makes predictions on new data. In the dataset used in this study, each website has 30 features and a corresponding class label.

The input to the model consists of multiple features related to websites. These features represent the website structurally and behaviorally. After preprocessing the dataset contains 30 features as mentioned. The models output is a binary class label and is as follows:

$0 \rightarrow$ phishing website

$1 \rightarrow$ represents a legitimate website.

The goal of the problem is to correctly classify the website as 0 or 1 using the given features. In other words the goal of the model is to be able to classify a new website as phishing or legitimate.To solve this problem, three different classification algorithms were used. These are Decision Tree, Naive Bayes, and k-Nearest Neighbors algorithms. These algorithms were trained tested and analyzed on the same dataset.

To measure the model performance, standard classification metrics such as accuracy, precision, recall, and F1-score were used. Additionally confusion matrices were analyzed to evaluate the models classification performance in more detail. Using these metrics the model's ability to detect phishing and secure websites was comprehensively evaluated.

Consequently this is a machine learning problem that uses 30 features specific to the website to classify it as either secure or phishing. The main goal is to increase correct classification minimize incorrect classification and achieve maximum accuracy.

B. Environment / Dataset Description

The dataset used in this report is a dataset prepared to detect phishing websites. This dataset was obtained from Kaggle and contains 30 features related to websites. The dataset consists of a total of 11055 sample websites. Each website is defined by different characteristics. These characteristics describe the website behaviorally and structurally.

In the dataset, the target variable is included for each sample website. The reason for this is to indicate whether each website is phishing or safe. In the original dataset, it is represented as 1 and -1. However, in this study, the data was converted to 1s and 0s to enable the model to function more effectively, making the dataset suitable for binary classification. As a result of this conversion, the value 0 represents phishing websites, while the value 1 represents safe websites.

Preprocessing steps were applied to the dataset. Therefore, the index column, which does not provide meaningful information for the model, was removed. Subsequently, the dataset was separated into input features X and the target variable Y.

For model training and evaluation, the dataset was divided into two parts: training and testing. 80% of the dataset was allocated for training, while 20% was used for testing. This allowed the model's performance on data it had not previously seen to be measured.

This dataset provides a suitable environment for evaluating the performance of machine learning algorithms, as it contains features commonly used for phishing detection.

Link of The Dataset: Dataset Link

IV. Methodology

A. Algorithm Selection

In this report, three different machine learning algorithms were selected to detect phishing websites (Decision Tree, Naive Bayes, K-Nearest Neighbors). These algorithms are methods that use different approaches for classification problems.

The Decision Tree algorithm is a method that models the decision making process by dividing data into branches according to specific rules. This algorithm is highly effective for classification problems thanks to its ability to capture complex relationships between data. Additionally the interpret ability of the model makes it much easier to determine which features are more effective for classification. For such reasons, the Decision Tree algorithm was chosen for this study.

The Naive Bayes algorithm is a probabilistic classification method based on the Naive Bayes Theorem. This algorithm probabilistically calculates which class a given data point will belong to based on specified features. It essentially estimates the class with the highest probability by using prior probability and conditional probability values. The most important assumption of this algorithm is that all features in the data set are independent of each other. This assumption simplifies the calculations considerably and enables the algorithm to operate quickly and efficiently even with high-dimensional data sets. Additionally, it is known for its low computational cost and fast training time, and is widely used especially in areas such as text classification.

The k-Nearest Neighbors algorithm is an instance-based learning method. It is a distance based method. That is it determines the class of a new data point by looking at its k nearest neighbors. It is based on the assumption that similar

data points belong to similar classes. It yields particularly effective results in situations where data distribution is crucial. In this study the k value was set to 5.

These three algorithms were selected in this study because they represent different learning approaches: Decision tree is a rule-based method that classifies data by dividing it according to specific decision rules. The Naive Bayes algorithm, on the other hand adopts a probabilistic approach and performs classification based on Bayes theorem. k-NN is a distance-based method and classifies according to the neighbors of the data point.

Since these algorithms employ different methods and principles, they enable learning from the same data set using different perspectives resulting in varied outcomes therefore an analysis is conducted on how each algorithm evaluates the features in the data set and under what conditions it yields more accurate results.

Additionally using different algorithms on the same data set helps us compare model performances and determine which algorithm would be more accurate for this type of classification problem thus enabling us to accurately reveal the advantages and limitations of the algorithms.

B. Implementation Details

The Python programming language was used in the execution of this study. Python was chosen due to the wide range of usable libraries it offers in the fields of data analysis and machine learning. The Python Pandas library was used for data processing and analysis tasks. Additionally the Scikit Learn (Sklearn) library was used to implement machine learning algorithms. Finally to visualize the results obtained from our analyses particularly for the graphical representation of confusion matrix results the Seaborn and Matplotlib libraries were used.

During the application process, the data set was first loaded in CSV format and basic analysis operations were performed. The index column which was not important for our model or meaningless to us, was removed from the data set. Subsequently adjustments were made to the result column which was the target variable for us. These adjustments involve converting -1 values to 0. The reason for doing this is to make the dataset suitable for a binary classification problem. After this our dataset becomes suitable for a binary classification problem.

The dataset is divided into input features (X) and the target variable (Y). Subsequently to train and test the model, the dataset is split using the train-test split method with 80% for training and 20% for test data. While performing this operation the random_state parameter is used to ensure that the results can be reproduced.

In this study three different machine learning algorithms were applied: Decision Tree, Naive Bayes, and k-Nearest Neighbors (k-NN). The Decision Tree algorithm uses a tree structure to divide the data into segments according to specific rules in order to learn relationships within the dataset and constructs a model based on this. The Naive Bayes algorithm performs probabilistic classification with the assumption of independence between features. The k-NN algorithm, on the other hand looks at each data point's nearest neighbors and assumes that it is in the same or a similar class as these neighbors in other words it performs classification based on the nearest neighbors and in this study the value of k was set to 5.

While analyzing the results obtained in our study various metrics were used to evaluate the performance of the models. These metrics are accuracy, precision, recall, and F1-score. With these metrics the overall success of the models was measured. In addition confusion matrices were calculated for each model and visualized and converted into graphs. This way the models true and false classification performances were included in our analyses in more detail.

In this study the results obtained by applying three algorithms representing different learning methods on the same data set were compared. Thus how different algorithm types perform on the same data set under identical conditions was analyzed and the differences between them were revealed. It has been determined which algorithm performs better and which algorithm provides more accurate results for this particular data set. This approach contributes to the identification and selection of the most suitable model.

C. Experimental Setup

In this study, the experimental setup was constructed based on the dataset used, algorithms employed, and evaluation metrics. All experiments used the same dataset and underwent the same preprocessing steps. Thus the performances of different algorithms on the same dataset were compared fairly.

The problem instances (test instances) used in the study consist of websites from the phishing dataset. The dataset was divided into two parts one for training the model and the other for testing. 80% of the total data was used as training data, while 20% was set aside for testing. This way, the models' performance was measured on the 20% portion they had not seen during training.

The parameter settings (parameters and settings) of the algorithms to be used in the experiments were left at standard values. The default parameters were used in the Decision Tree algorithm. For the Naive Bayes algorithm, the Gaussian Naive Bayes model was chosen, and in the k-Nearest Neighbors Algorithm, the number of neighbors (k) was set to 5. This choice was made to ensure a balanced measurement and evaluation of the algorithms' performance.

During the experiments, the Python programming language was used, and the scikit-learn library was utilized in the model development process. Additionally, the pandas library was employed for data processing operations. For visualization, the seaborn and matplotlib libraries were used. A standard personal computer was used as the testing environment. No special hardware or high performance computing infrastructure was required for the study. This demonstrates that the experiment conducted and the methods used are also applicable with low hardware requirements. Thanks to this experimental setup different machine learning algorithms could be compared on

the same data set thus increasing the reliability of the obtained results.

V. Results

A. Evaluation Metrics

In this study, four types of metrics were used to evaluate the performance of classification models: accuracy, precision, recall, and F1-score. Instead of relying on a single metric evaluating based on four different metrics allowed us to analyze the model's performance from various perspectives.

Accuracy, shows us the ratio of correct predictions among the total predictions made by the model. It provides information about overall performance. However it is not a sufficient evaluation criterion on its own. Therefore we also need other metrics.

Precision, represents the proportion of truly positive cases among those predicted positive by the model. This metric plays a critical role especially in situations where false positive predictions are important. Recall, shows us how many of the true positive examples were correctly identified by the model which is important for us in terms of detecting and understanding the positive examples the model missed. F1-score evaluates model performance in more detail by giving us a balanced average of precision and recall metrics. It enables the model to obtain much more meaningful results, especially in cases where there is imbalanced distribution among classes in the dataset.

In addition to these metrics, a confusion matrix was used to analyze the model's performance in more detail. Through these matrices true positive, true negative, false positive, and false negative values can be examined separately. The confusion matrix, along with the overall accuracy value reveals what types of errors the model makes. This way when our model distinguishes between websites, the websites are analyzed in more depth. With this approach the advantages and disadvantages of the models can be determined more clearly.

B. Results Summary

Algorithm	Accuracy (%)	Precision (Weighted avg)	Recall (Weighted avg)	F1-score (Weighted avg)
Decision Tree	95.75%	0.96	0.96	0.96
k-NN	93.6%	0.94	0.94	0.94
Naive Bayes	58.2%	0.79	0.58	0.53

Table 1: Algorithm Performance Comparison

The results of the performance of all algorithms are shown in Table 1. According to the results the Decision Tree Algorithm achieved higher performance than the other two algorithms in terms of all metrics, with an accuracy rate of 95.75%. Additionally, the Decision Tree produced balanced and high results in precision, recall, and F1-score values.The k-Nearest Neighbors (k-NN) algorithm, on the other hand, demonstrated strong performance with a 93.6% accuracy rate. However, its results were approximately 2% lower than those of the Decision Tree algorithm.In contrast the Naive Bayes algorithm performed quite poorly compared to other algorithms with an accuracy rate of 58.2%. The primary reason for this is that this algorithm uses the independence assumption among features and this assumption is not fully satisfied in the data set we used. When we make a general evaluation based on the results it can be easily stated that the Decision Tree algorithm is the most suitable for this classification problem. On the other hand it was concluded that the Naive Bayes algorithm showed significantly lower performance compared to Decision Tree and k-NN.

C. Visualizations

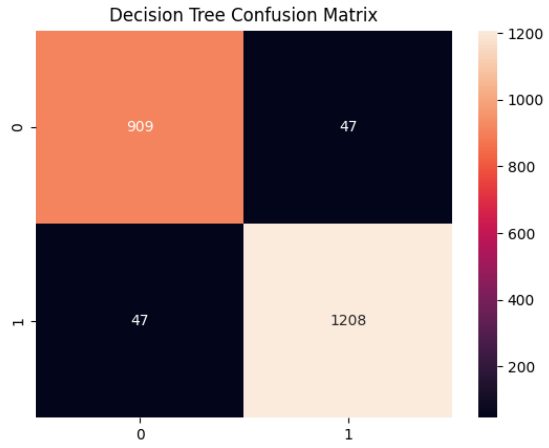


Figure 1: Decision Tree

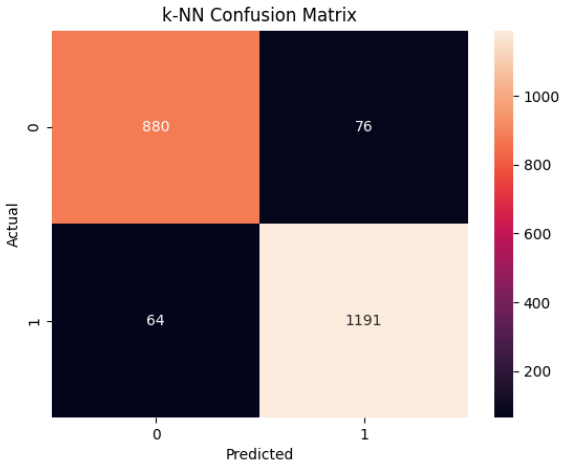


Figure 2: k-NN

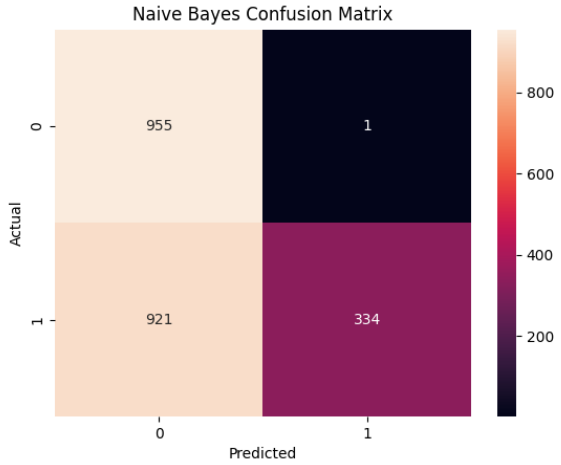


Figure 3: Naive Bayes

VI. Discussion

In this study we conducted, the Decision Tree Algorithm showed the best performance. It has the highest accuracy rate across all metrics. This result indicates that this algorithm is capable of accurately modeling the complex relationships in this dataset. Thanks to its tree structure, the data is divided according to certain rules and classified in a more meaningful way. Therefore, this algorithm has been determined as the most logical and successful method for this dataset.

Of course like most studies, this study also has certain limitations. First, the data set used was obtained from a single source, and no work has been done or testing performed on different data sets. Additionally the parameters of the algorithms were left at their default values and no hyperparameter tuning was performed. Therefore some algorithms may not have operated at their full potential. Furthermore data preprocessing steps were kept limited.

When the results obtained from our study are analyzed, the most striking finding is that the Naive Bayes Algorithm performs significantly worse than other algorithms. The reason for this is that the features in the columns of the dataset are closely interrelated. Since this algorithm is based on the independence assumption its performance on such datasets is significantly poor. However another noteworthy result is the Decision Tree Algorithm's very high and balanced performance.

The main takeaways from this project are that different machine learning algorithms can produce significantly different results on the same dataset. Additionally, it has become clear that simply looking at accuracy is not sufficient for us metrics such as precision, recall, and F1-score must also be included in our analyses. Moreover it is clearly evident that the structure of the data set and the relationship between its features are among the issues that require the utmost attention when selecting an algorithm. This study reveals how crucial it is to select the right algorithm in terms of model performance and data analysis. Therefore it is also evident how critical algorithm selection is for user security.

VII. Conclusion and Future Work

In this study, different machine learning algorithms were applied for the detection of phishing websites, and their performances were compared with each other. The performance of the Decision Tree, Naive Bayes, and k-Nearest Neighbors (k-NN) algorithms was measured using classification metrics such as accuracy, precision, recall rate, and F1 score.

According to our results, the Decision Tree Algorithm showed the most successful performance. The k-NN algorithm, although achieving a high accuracy rate, fell short of the Decision Tree algorithm. The Naive Bayes algorithm, on the other hand, demonstrated very low performance. The primary reason for this is that the independence assumption between features in the data set is not compatible with this algorithm's assumption.

This study demonstrates how crucial it is to select the appropriate algorithm for the structure of the data set in terms of performance. Since algorithm selection directly impacts model performance, it plays a critical role in the success of phishing detection systems.

In future studies hyperparameter adjustments can be made to improve model performance. Additionally algorithms can be tested on different datasets to increase the generalizability of the results. Furthermore higher performance can be achieved using deep learning approaches.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Pearson, 2022.

[2] D. Sahoo, C. Liu, and S. C. Hoi, "Malicious URL Detection using Machine Learning: A Survey," https://arxiv.org/abs/1701.07179

[3] F. Salahdine, Z. El Mrabet, and N. Kaabouch, "Phishing Attacks Detection – A Machine Learning-Based Approach," https://arxiv.org/abs/2201.10752

[4] [https://www.kaggle.com/datasets/akashkr/phishing-website-dataset]

Appendix

Source code available at: Source Code of the Project